CN-Stats-Lec12-Statistical Test for Bivariate Analysis

Statistical Test for Bivariate Analysis

🎯 Session Objectives:

- ✅ Perform and interpret One-Sample T-Tests, ANOVA, and Chi-Square Tests using Python.
- ✅ Understand the purpose and application of the T-test, ANOVA, and Chi-Square tests in data analysis.
- ✅ Formulate and test hypotheses using the T-test to compare means between two groups.
- ✅ Apply ANOVA to compare means across multiple groups and assess the relationship between continuous and categorical variables.
- ✅ Conduct Chi-Square tests to examine associations between categorical variables.
- ✅ Interpret the results of these statistical tests and make data-driven decisions based on the findings.

From T to F

The F-statistic extends the t-statistic concept to multiple groups. While t-statistic compares two means, F-statistic compares variability between multiple group means to variability within groups.

The Formula

$$F = \frac{\text{Variation Between Groups}}{\text{Variation Within Groups}}$$

A large F-statistic suggests group means are significantly different compared to within-group variability.

```
Degree of Freedom -> Between  [k-1]
Degree of Freedom -> Within   [n-k]
```

```python
# Degree of Freedom -> Between  [k-1] # (3-1) # 2
# Degree of Freedom -> Within   [n-k] # (15-3) # 12

# ANOVA Example: Energy Drink Analysis
import scipy.stats as stats

Drink_A = [105, 110, 98, 107, 103]
Drink_B = [130, 125, 132, 128, 129]
Drink_C = [120, 115, 117, 119, 121]

f_stats , p_value = stats.f_oneway(Drink_A, Drink_B, Drink_C)
print(f"Annova - F_Emperical: {f_stats}")
print(f"Probability Value: {p_value}")
```

```
Annova - F_Emperical: 67.39634146341461
Probability Value: 2.9844274607194716e-07
```

```
F_Emperical > F_Critical [Low p_value] ~ Reject Null Hypothesis.
```

```
degree_of_freedom = 2 [Numerator] => Between Group (k-1) -> 3-1 = 2
degree_of_freedom = 12 [Denominator] => Within Group (n-k) -> 15-3 = 12

95% Confidence Level -> [alpha = 0.05] -> df(2,12) -> 3.885
99% Confidence Level -> [alpha = 0.01] -> df(2,12) -> 6.927

Emperical_F_Stats > Critical_F_Stats =>
67.39 > (3.885 , 6.927)
Probability Value: 2.9844274607194716e-07 ~ 0.000000298 < 0.05
# Reject the Null Hypothesis and Accepting the Alternative Hypothesis
```

```python
# Chi-Square Test : Testing Relationships Between Categories
import scipy.stats as stats
import numpy as np

observed = np.array([
    [50,30,20],
    [30,40,30],
    [20,30,50]
])

chi2_stat, p_value, degrees_of_freedom, expected_frequencies = stats.chi2_contingency(observed)
chi2_stat, p_value, degrees_of_freedom, expected_frequencies
```

```
(30.0,
 4.894437128029217e-06,
 4,
 array([[33.33333333, 33.33333333, 33.33333333],
        [33.33333333, 33.33333333, 33.33333333],
        [33.33333333, 33.33333333, 33.33333333]]))
```

```
4.894437128029217e-06 = 0.00000489 < (0.05) : Reject Null Hypothesis
Chi Square Test
Emperical_Value 30 > [9.488,11.143,13.277,14.860]
Accepting the Alternative Hypothesis : Which Signifies that there is a correlation exist
between Products and the Age Group.
```

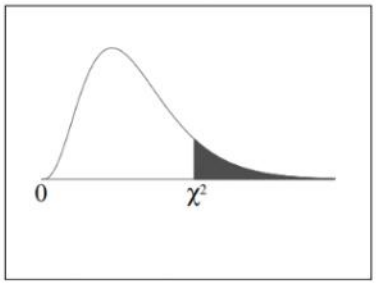
This means different age groups show distinct preferences for different product types. The company should consider age-specific marketing strategies.

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

χ^2 = the test statistic $\sum$ = the sum of

O = Observed frequencies E = Expected frequencies

Chi-Square Distribution Table



The shaded area is equal to α for $\chi^2 = \chi^2_\alpha$. 95% Confidence

df	$\chi^2_{.995}$	$\chi^2_{.990}$	$\chi^2_{.975}$	$\chi^2_{.950}$	$\chi^2_{.900}$	$\chi^2_{.100}$	$\chi^2_{.050}$	$\chi^2_{.025}$	$\chi^2_{.010}$	$\chi^2_{.005}$
1	0.000	0.000	0.001	0.004	0.016	2.706	3.841	5.024	6.635	7.879
2	0.010	0.020	0.051	0.103	0.211	4.605	5.991	7.378	9.210	10.597
3	0.072	0.115	0.216	0.352	0.584	6.251	7.815	9.348	11.345	12.838
4	0.207	0.297	0.484	0.711	1.064	7.779	9.488	11.143	13.277	14.860
5	0.412	0.554	0.831	1.145	1.610	9.236	11.070	12.833	15.086	16.750

Observed (30) > [9.488,11.143,13.277,14.860]

```python
# Statistical Tests for Bivariate Analysis
# Flight Delay Dataset
import pandas as pd

flight_delay = pd.read_csv("flight_delays_train.csv")
flight_delay
```

	Month	DayofMonth	DayOfWeek	DepTime	UniqueCarrier	Origin	Dest	Distance	dep_delayed_15min
0	c-8	c-21	c-7	1934	AA	ATL	DFW	732	N
1	c-4	c-20	c-3	1548	US	PIT	MCO	834	N
2	c-9	c-2	c-5	1422	XE	RDU	CLE	416	N
3	c-11	c-25	c-6	1015	OO	DEN	MEM	872	N
4	c-10	c-7	c-6	1828	WN	MDW	OMA	423	Y
...	...	...	...	...	...	...	...	...	...
99995	c-5	c-4	c-3	1618	OO	SFO	RDD	199	N
99996	c-1	c-18	c-3	804	CO	EWR	DAB	884	N
99997	c-1	c-24	c-2	1901	NW	DTW	IAH	1076	N
99998	c-4	c-27	c-4	1515	MQ	DFW	GGG	140	N
99999	c-11	c-17	c-4	1800	WN	SEA	SMF	605	N

100000 rows × 9 columns

```
DepTime : 1934 -> 24 hrs Format -> 7:34PM
1548 : 3:48PM
```

```
flight_delay['Month'].unique()
```

```
array(['c-8', 'c-4', 'c-9', 'c-11', 'c-10', 'c-1', 'c-7', 'c-6', 'c-5',
       'c-3', 'c-12', 'c-2'], dtype=object)
```

```
flight_delay['Month'].value_counts()
```

```
Month
c-8     8830
c-7     8706
c-3     8595
c-5     8543
c-6     8414
c-4     8408
c-10    8405
c-12    8265
c-11    8178
c-9     8163
c-1     8075
c-2     7418
Name: count, dtype: int64
```

```
# C-1 : Jan till C-12 : Dec
```

```
flight_delay.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 9 columns):
 #   Column             Non-Null Count   Dtype
---  ------             --------------   -----
 0   Month              100000 non-null  object
 1   DayofMonth         100000 non-null  object
 2   DayOfWeek          100000 non-null  object
 3   DepTime            100000 non-null  int64
 4   UniqueCarrier      100000 non-null  object
 5   Origin             100000 non-null  object
 6   Dest               100000 non-null  object
 7   Distance           100000 non-null  int64
 8   dep_delayed_15min  100000 non-null  object
dtypes: int64(2), object(7)
memory usage: 6.9+ MB
```

```
flight_delay['DayofMonth'].unique()
```

```
array(['c-21', 'c-20', 'c-2', 'c-25', 'c-7', 'c-3', 'c-27', 'c-29',
       'c-28', 'c-5', 'c-6', 'c-10', 'c-19', 'c-26', 'c-14', 'c-22',
       'c-9', 'c-31', 'c-4', 'c-16', 'c-18', 'c-30', 'c-1', 'c-24',
       'c-15', 'c-17', 'c-8', 'c-12', 'c-13', 'c-11', 'c-23'],
      dtype=object)
```

```
len(flight_delay['DayofMonth'].unique())
```

```
31
```

```python
flight_delay['DayOfWeek'].value_counts()
```

```
DayOfWeek
c-4    14736
c-5    14657
c-2    14604
c-1    14569
c-3    14508
c-7    14097
c-6    12829
Name: count, dtype: int64
```

```
DayOfWeek : C-1 : Mon till C-7 : Sun
```

```python
flight_delay['UniqueCarrier'].value_counts()
```

```
UniqueCarrier
WN    15082
AA     9418
DL     8128
MQ     7443
OO     7390
UA     6876
US     6482
NW     6403
XE     5901
OH     4594
CO     4334
EV     3930
FL     3039
AS     2222
YV     2128
B6     1838
HP     1378
F9     1006
DH      966
HA      762
TZ      446
AQ      234
Name: count, dtype: int64
```

The **IATA code "WN"** is associated with **Southwest Airlines**, which is the world's largest low-cost carrier. The code was chosen due to its availability, not as a reference to a famous person, as a popular urban legend suggests. University of Washington

Flightnetwork
in-en.flightnetwork.com › american (aa)
Book Now
American (AA) Flights | Fly American (AA)
Sponsored Book cheap **American** (**AA**) **flights** on Flightnetwork. Save up to 60%. Save up to 60% on your next **flight**. Compare all **American** (**AA**) **flights**
Site visitors: Over 100K in the past month

```
flight_delay['dep_delayed_15min'].value_counts()
```

```
dep_delayed_15min
N    80956
Y    19044
Name: count, dtype: int64
```

```
flight_delay['DepTime'] = pd.to_numeric(flight_delay['DepTime'] , errors = 'coerce')
flight_delay['Distance'] = pd.to_numeric(flight_delay['Distance'] , errors = 'coerce')

# T-Test [Grouping : Split the dataset into two groups]
delayed_flights = flight_delay[flight_delay['dep_delayed_15min'] == 'Y']
non_delayed_flights = flight_delay[flight_delay['dep_delayed_15min'] == 'N']
```

```
delayed_flights
```

	Month	DayofMonth	DayOfWeek	DepTime	UniqueCarrier	Origin	Dest	Distance	dep_delayed_15min
4	c-10	c-7	c-6	1828	WN	MDW	OMA	423	Y
18	c-10	c-14	c-5	1704	OO	SLC	SEA	689	Y
21	c-6	c-21	c-3	1519	EV	ATL	ILM	377	Y
33	c-12	c-3	c-7	1653	EV	ATL	MYR	317	Y
35	c-3	c-24	c-4	2303	US	CLT	CHS	168	Y
...	...	...	...	...	...	...	...	...	...
99971	c-7	c-31	c-1	2027	DL	ATL	SJC	2116	Y
99976	c-5	c-20	c-5	2117	MQ	LGA	BGR	378	Y
99980	c-4	c-28	c-4	2222	XE	EWR	DAB	884	Y
99989	c-4	c-11	c-1	1730	WN	JAN	BWI	888	Y
99993	c-9	c-16	c-6	722	XE	CHS	IAH	925	Y

19044 rows × 9 columns

```
non_delayed_flights
```

	Month	DayofMonth	DayOfWeek	DepTime	UniqueCarrier	Origin	Dest	Distance	dep_delayed_15min
0	c-8	c-21	c-7	1934	AA	ATL	DFW	732	N
1	c-4	c-20	c-3	1548	US	PIT	MCO	834	N
2	c-9	c-2	c-5	1422	XE	RDU	CLE	416	N
3	c-11	c-25	c-6	1015	OO	DEN	MEM	872	N
5	c-8	c-3	c-4	1918	NW	MEM	MCO	683	N
...	...	...	...	...	...	...	...	...	...
99995	c-5	c-4	c-3	1618	OO	SFO	RDD	199	N
99996	c-1	c-18	c-3	804	CO	EWR	DAB	884	N
99997	c-1	c-24	c-2	1901	NW	DTW	IAH	1076	N
99998	c-4	c-27	c-4	1515	MQ	DFW	GGG	140	N
99999	c-11	c-17	c-4	1800	WN	SEA	SMF	605	N

80956 rows × 9 columns

```
# T-Test For Depart Time w.r.t dep_delayed_15min
from scipy.stats import ttest_ind
t_stats , p_value = ttest_ind(delayed_flights['DepTime'],non_delayed_flights['DepTime'])
print(f"T-Test : {t_stats}")
print(f"Probability Value : {p_value:.2}")
```

```
T-Test : 79.50072484951748
Probability Value : 0.0
```

```
# T-Test For Distance w.r.t dep_delayed_15min
from scipy.stats import ttest_ind
t_stats , p_value = ttest_ind(delayed_flights['Distance'],non_delayed_flights['Distance'])
print(f"T-Test : {t_stats}")
print(f"Probability Value : {p_value:.2}")
```

```
T-Test : -0.8646551177928152
Probability Value : 0.39
```